KAUNO TECHNOLOGIJOS UNIVERSITETAS

INFORMATIKOS FAKULTETAS

PROGRAMŲ INŽINERIJOS KATEDRA

P170B400 Algoritmų sudarymas ir analizė

Laboratorinis darbas 1

Studentas:

Ignas Vasiliauskas IFF-4/1

Dėstytojas:

Makackas Dalius
Pilkauskas Vytautas

Kaunas, 2026

Turinys

1. Įvadas 3
2. Užduoties aprašymas 4
3. Programos rezultatai 6
4. Grafikai 7
5. Rekurentinės lygtys 9
6. Sprendimas 13

1. Įvadas

Šis laboratorinis darbas yra skirtas sukurti efektyvias rekursinių procedūrų realizacijas tam tikrai užduočiai, eksperimentiškai išanalizuoti jų veikimo greitį, sudaryti ir išspręsti rekurentines lygtis.

Pagrindinis tikslas – sukurti fraktalo generavimo rekursinę procedūrą, teoriškai ir praktiškai įvertinti jos veikimo efektyvumą.

1. Sukurti dvi procedūrų versijas:
    - Pirmoj versijoj nurodome rekursijos gylį.
    - Antroj versijoj nurodome generuojamo paveikslėlio matmenis.
2. Abiejų procedūrų rezultatus išsaugoti kaip .BMP formato failus.
3. Eksperimentiškai išmatuoti procedūrų veikimo laiką ir atliktų operacijų skaičių abiem atvejais, rezultatus pateikti grafikuose.
4. Sudaryti rekurentines lygtis kiekvienai procedūros realizacijai, jas išspręsti ir palyginti gautus teorinius rezultatus su eksperimentiniais duomenimis.

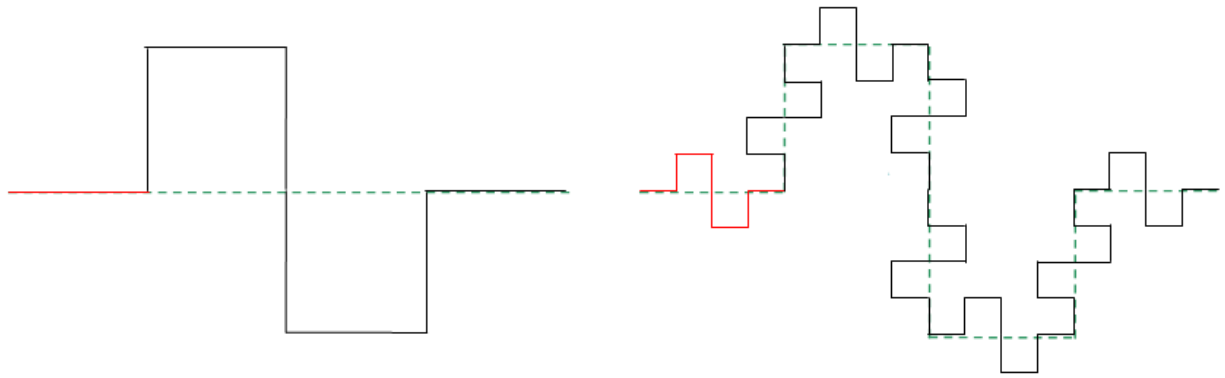
Šis darbas leidžia giliau suprasti rekursinių algoritmų veikimą, jų teorinį pagrindą ir praktinį pritaikymą fraktalų generavimo užduotyse.

2. Užduoties aprašymas

Užduotis: Parašyti efektyvias rekursinių procedūrų realizacijas **(visi procedūrų ciklai realizuojami rekursinių iškvietimų pagalba)**, nenaudojant jokių grafinių bibliotekų, kurios generuotų nurodytos struktūros BMP formato paveikslėlį (1 LD individualios užduoties pasirinkimo teste pateiktas konkretus variantas).

Reikalavimai:

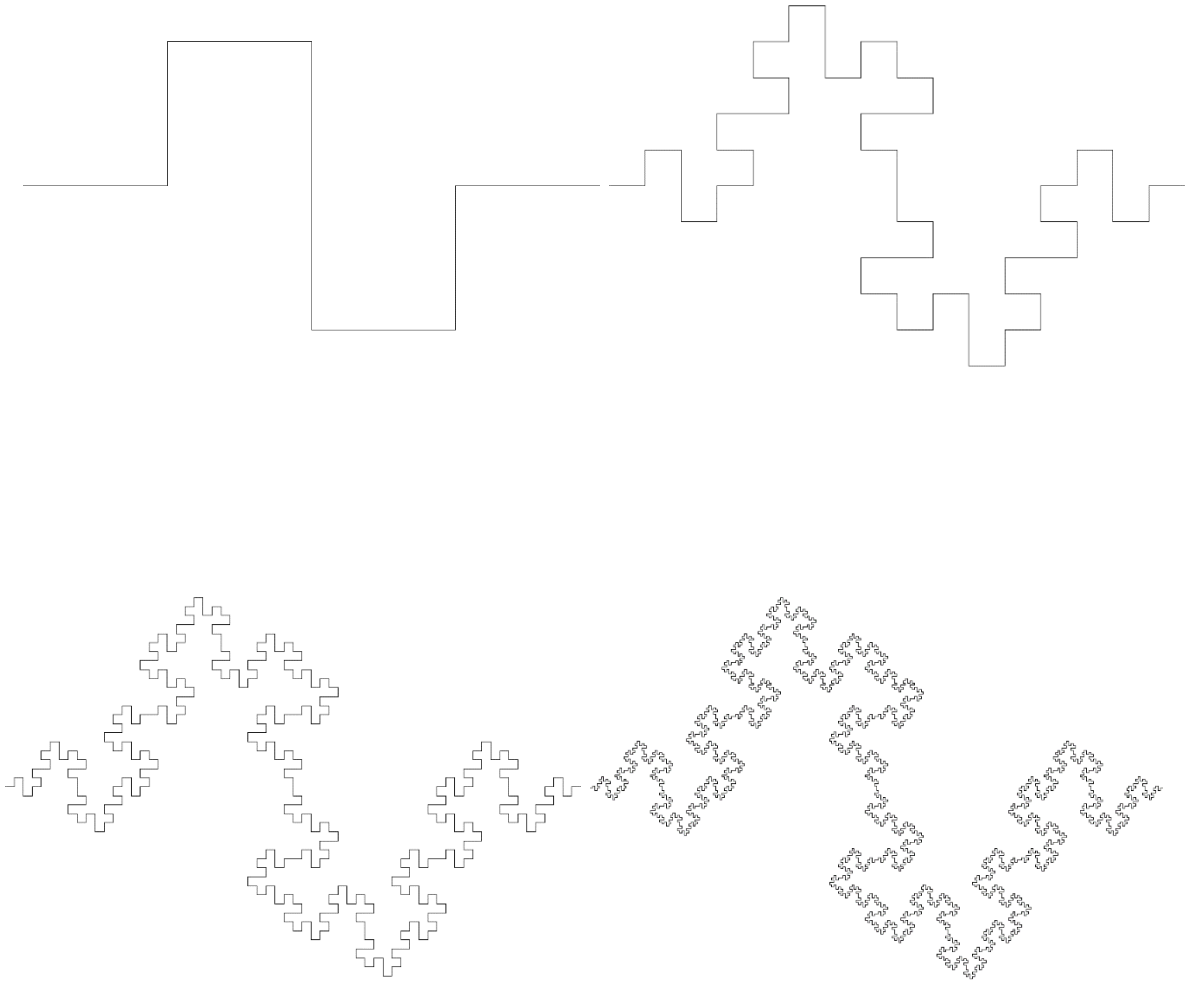
- Du realizacijos atvejai: 1) Nurodomas rekursijos gylis; 2) Generuojamas maksimalaus detalumo paveikslėlis, pasirinktiems paveikslėlio matmenims.
- Programų rezultatas BMP formato bylos demonstruojančios programų veikimą. (3 balai)
- Eksperimentiškai nustatykite darbo laiko ir veiksmų skaičiaus priklausomybę nuo rekursijos gylio ar generuojamo paveikslėlio dydžio (taškų skaičiaus). Gautus rezultatus atvaizduokite grafikais (4 grafikai). Grafiką turi sudaryti nemažiau kaip 5 taškai ir paveikslėlio taškų skaičius turi didėti proporcingai (kartais). Paskutinis eksperimento grafiko taškas turi atitikti maksimalų galimą paveikslėlio raišką (apie 20000x20000 taškų). (2 balas)
- Analitiškai įvertinkite procedūrų, kurios generuoja paveikslėlius, veiksmų skaičių ir laiką, sudarydami rekurentines lygtis ir jas išspręskite. Gauti rezultatai turi patvirtinti eksperimentinius rezultatus (našumo testus: vykdymo laiką ir veiksmų skaičių). (2 balas)
- Paruoškite detalią ataskaitą su atliktais skaičiavimais. Ataskaita privalo tenkinti rašto darbų reikalavimus (visos formulės privalo būti rašomos pagal matematikoje priimtus kanonus, programos fragmentai neturėtų būti tiesiog „blynai“ (ekrano kopijos...) Rekomenduojama naudoti Latex ar Word redaktorius matematinių išraiškų rašymui. (3 balai)



1 pav. užduoties variantas

BMP paveikslėlio spalvų gylis: 24 bitai

3. Programos rezultatai

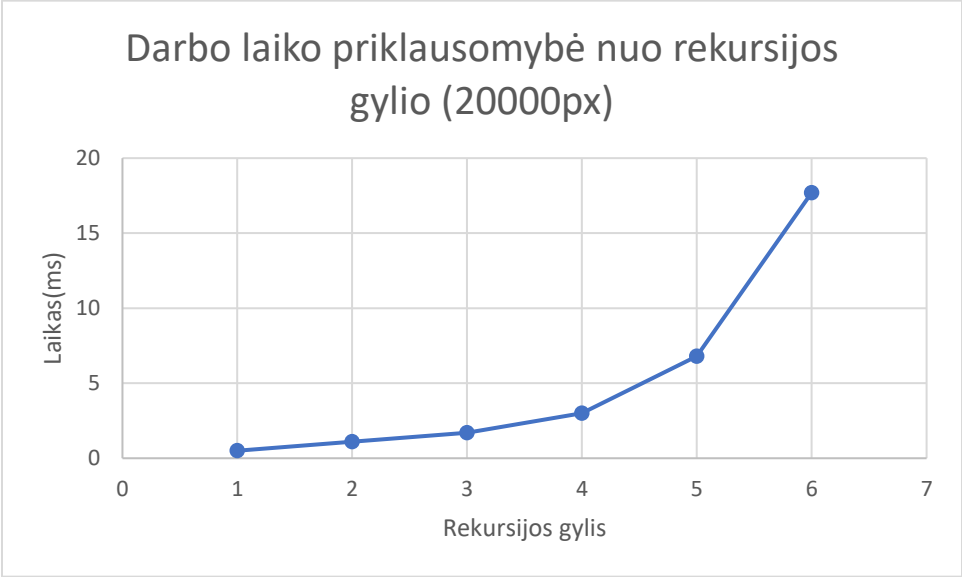


2 pav. Programos rezultatai

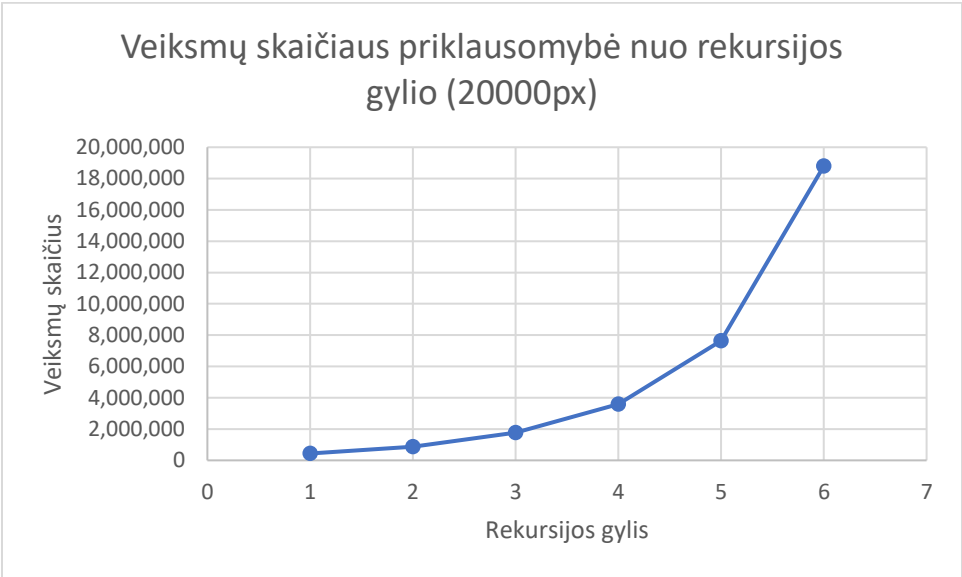
Programos rezultatai sutampa su užduoties variantu.

4. Grafikai

Priklausomybės nuo rekursijos gylio grafikai:

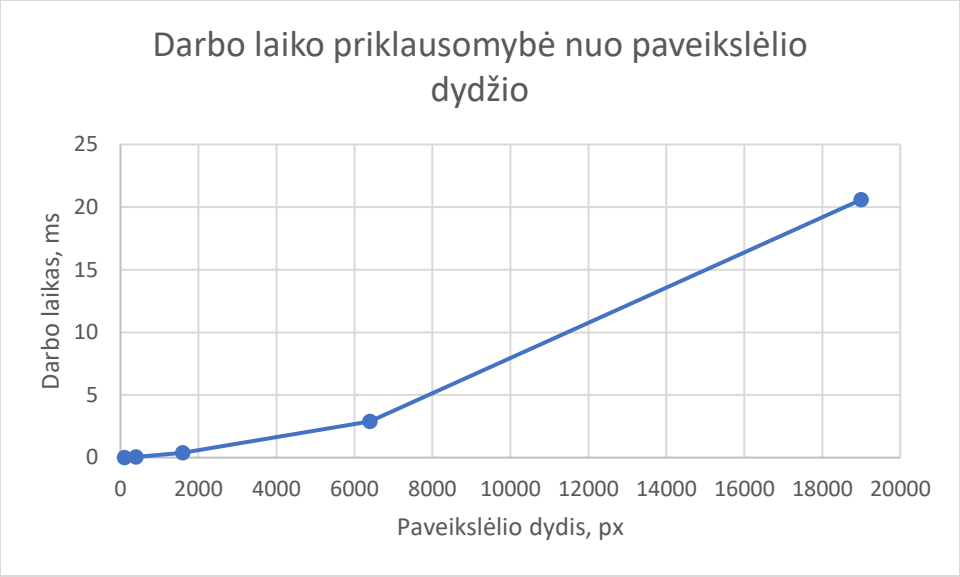


3 pav. Darbo laiko priklausomybė nuo rekursijos gylio

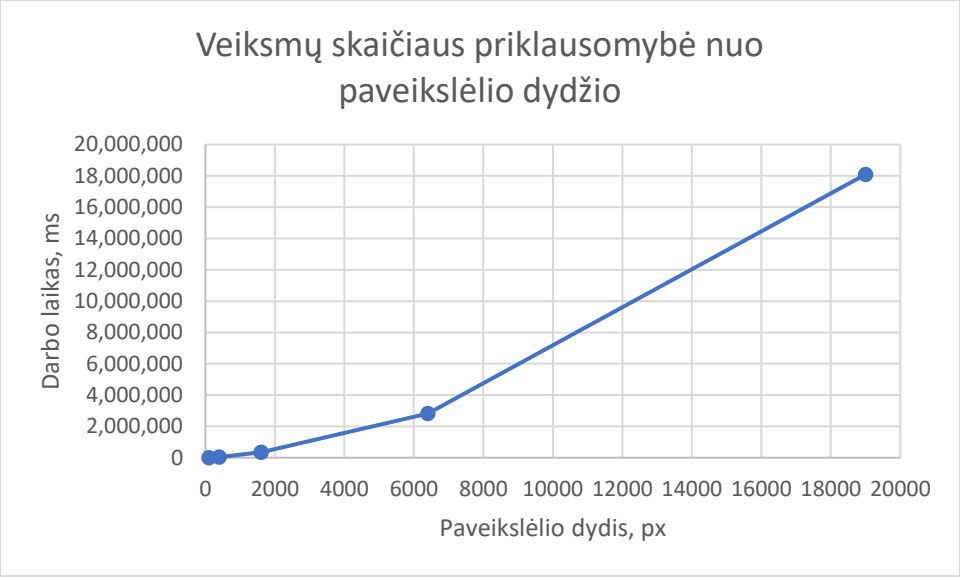


4 pav. Veiksmų skaičius priklausomybė nuo rekursijos gylio

Priklausomybės nuo paveikslėlio dydžio grafikai:



5 pav. Darbo laiko priklausomybė nuo paveikslėlio dydžio



6 pav. Veiksmų skaičiaus priklausomybė nuo paveikslėlio dydžio

5. Rekurentinės lygtys

SetPixel metodas

Metodas nustato vieno pikselio reikšmę BMP duomenų masyve.

Kodas	Kaina	Kiekis
`static void SetPixel(int x, int y, byte r, byte g, byte b, byte[] data)`		
`{`		
`if (x < 0 || x >= resolution ||`	c_1	1
`y < 0 || y >= resolution) return;`		
`int flippedY = resolution - 1 - y;`	c_2	$1 - \chi_1$
`int idx = (flippedY * resolution + x) * 3;`	c_3	$1 - \chi_1$
`data[idx] = b;`	c_4	$1 - \chi_1$
`data[idx + 1] = g;`	c_5	$1 - \chi_1$
`data[idx + 2] = r;`	c_6	$1 - \chi_1$
`}`		

kur $\chi_1 \in [0, 1]$ — ribų patikrinimo rezultatas (1, jei pikselis už ribų, 0 — priešingu atveju).

$$f(x, y, r, g, b, data) = c_1 + (1 - \chi_1)(c_2 + c_3 + c_4 + c_5 + c_6)$$

Asimptotinis sudėtingumas: $f = O(1)$

DrawLine metodas

Piešia horizontalią arba vertikalią liniją. Tegu L — linijos ilgis pikseliais. $S = O(1)$ — SetPixel kaina.

Kodas	Kaina	Kiekis
`static void DrawLine(int x0, int y0, int x1, int y1,`		
`byte r, byte g, byte b, byte[] data)`		

Kodas	Kaina	Kiekis
`{`		
`if (x0 == x1)`	c_1	1
`for (int y = Math.Min(y0, y1);`	c_2	$\chi_1 \cdot 1$
`y <= Math.Max(y0, y1);`	c_3	$\chi_1 \cdot (L + 1)$
`y++)`	c_4	$\chi_1 \cdot L$
`SetPixel(x0, y, r, g, b, data);`	S	$\chi_1 \cdot L$
`else`		
`for (int x = Math.Min(x0, x1);`	c_5	$(1-\chi_1) \cdot 1$
`x <= Math.Max(x0, x1);`	c_6	$(1-\chi_1)(L + 1)$
`x++)`	c_7	$(1-\chi_1) \cdot L$
`SetPixel(x, y0, r, g, b, data);`	S	$(1-\chi_1) \cdot L$
`}`		

$$f(L) = c_1 + \chi_1(c_2 + c_3(L+1) + c_4 \cdot L + S \cdot L)$$
$$+ (1-\chi_1)(c_5 + c_6(L+1) + c_7 \cdot L + S \cdot L)$$

Asimptotinis sudėtingumas: $f(L) = O(L)$

DrawRecursive metodas

Tegu n = depth, $\chi_1 \in \{0, 1\}$ — 1, jei lapinis mazgas (depth == 0); 0 kitu atveju.

Kodas	Kaina	Kiekis
`static void DrawRecursive(double x0, double y0,`		

Kodas	Kaina	Kiekis
double x8, double y8, int depth, byte[] data)		
{		
if (depth == 0)	c_1	$1, \quad \chi_1 \in [0,1]$
{		
DrawLine(	$D(L)$	χ_1
(int)Math.Round(x0), (int)Math.Round(y0),	c_2	χ_1
(int)Math.Round(x8), (int)Math.Round(y8),	c_2	χ_1
0, 0, 0, data);		
return;	c_3	χ_1
}		
double dx = (x8 - x0) / 4.0;	c_4	$1 - \chi_1$
double dy = (y8 - y0) / 4.0;	c_5	$1 - \chi_1$
double px = dy;	c_6	$1 - \chi_1$
double py = -dx;	c_7	$1 - \chi_1$
double x1 = x0 + dx, y1 = y0 + dy;	c_8	$1 - \chi_1$
double x2 = x1 + px, y2 = y1 + py;	c_9	$1 - \chi_1$
double x3 = x2 + dx, y3 = y2 + dy;	c_{10}	$1 - \chi_1$

Kodas	Kaina	Kiekis
`double x4 = x3 - px, y4 = y3 - py;`	c_{11}	$1 - \chi_1$
`double x5 = x4 - px, y5 = y4 - py;`	c_{12}	$1 - \chi_1$
`double x6 = x5 + dx, y6 = y5 + dy;`	c_{13}	$1 - \chi_1$
`double x7 = x6 + px, y7 = y6 + py;`	c_{14}	$1 - \chi_1$
`DrawRecursive(x0, y0, x1, y1, depth - 1, data);`	$T(n-1)$	$1 - \chi_1$
`DrawRecursive(x1, y1, x2, y2, depth - 1, data);`	$T(n-1)$	$1 - \chi_1$
`DrawRecursive(x2, y2, x3, y3, depth - 1, data);`	$T(n-1)$	$1 - \chi_1$
`DrawRecursive(x3, y3, x4, y4, depth - 1, data);`	$T(n-1)$	$1 - \chi_1$
`DrawRecursive(x4, y4, x5, y5, depth - 1, data);`	$T(n-1)$	$1 - \chi_1$
`DrawRecursive(x5, y5, x6, y6, depth - 1, data);`	$T(n-1)$	$1 - \chi_1$
`DrawRecursive(x6, y6, x7, y7, depth - 1, data);`	$T(n-1)$	$1 - \chi_1$
`DrawRecursive(x7, y7, x8, y8, depth - 1, data);`	$T(n-1)$	$1 - \chi_1$
`}`		

Nelapiniame mazge ($1 - \chi_1 = 1$):

$$f = c_1 + c_4 + c_5 + c_6 + c_7 + c_8 + c_9 + c_{10} + c_{11} + c_{12} + c_{13} + c_{14} + 8 \cdot T(n-1) = c + 8 \cdot T(n-1)$$

Lapiniame mazge ($\chi_1 = 1$):

$$f = c_1 + D(L) \approx O(1)$$

6. Sprendimas

$$T(n) = 8 \cdot T(n-1) + c, \quad T(0) = k$$

kur c — fiksuotos operacijos nelapiniame mazge; k — DrawLine kaina baziniame lygyje

Pakartotinio pakeitimo metodas:

$$T(n) = 8 \cdot T(n-1) + c$$

$$= 8(8T(n-2) + c) + c = 8^2 T(n-2) + c(8 + 1)$$

$$= 8^2(8T(n-3) + c) + c(8+1) = 8^3 T(n-3) + c(8^2 + 8 + 1)$$

$$\vdots$$

$$= 8^n T(0) + c(8^{n-1} + \cdots + 8 + 1) = k \cdot 8^n + c \cdot \Sigma_{=0}^{k}{}^{n-1} 8^k$$

$$= k \cdot 8^n + c \cdot (8^n - 1) / 7$$

$$T(n) = (k + c/7) \cdot 8^n - c/7$$

Asimptotinis sudėtingumas (pagal gylį n): $T(n) = O(8^n)$